

Analysis and Design of Algorithms

Project II

Problem Statement

Real-World Narrative

A university mentorship program runs weekly one-on-one sessions between fellows and mentors. Each week, fellows may submit a request for a session with a mentor to address a technical challenge in their project.

Mentors are domain specialists, each covering a small number of topics. A request can only be fulfilled by a mentor whose specialties include the requested topic. Because the number of available slots is limited each week, and because not all topics have equal mentor coverage, some fellows may go unserved in a given week. The program lead requires an automated weekly assignment that respects topic constraints, prioritizes urgent requests, and prevents any fellow from being repeatedly skipped over the lifetime of the cohort.

Notation

Let the following symbols define the scale of the problem each week:

- F : number of fellows enrolled in the program.
- M : number of available mentors.
- S : number of slots offered this week, where each slot is a `(mentor_id, time_block)` pair with capacity one.
- R : number of requests submitted this week, where $R \leq F$.

Input Definition

At the start of each week, the system receives:

- **Fellows:** a set of F fellows, each identified by a unique fellow ID.
- **Mentors:** a set of M mentors, each identified by a unique mentor ID and associated with a set of one or more specialty topics.

- **Slots:** a set of S slots for the current week. Each slot is a (`mentor_id`, `time_block`) pair with a capacity of exactly one fellow.
- **Requests:** a set of $R \leq F$ requests, at most one per fellow. Each request is a tuple:

(`fellow_id`, [`available_slot_ids`], `requested_topic`, `urgency`, `timestamp`)

where `urgency` $\in$ {`blocker`, `normal`, `exploratory`} and `timestamp` records when the request was submitted.

System State

The following variable is maintained by the system across weeks and is not part of the weekly input:

- s_f : the number of consecutive weeks fellow f has gone without an assignment. Initialized to zero for all fellows at the start of the program. After each weekly run, s_f is updated as follows:

$$s_f \leftarrow \begin{cases} 0 & \text{if fellow } f \text{ was assigned this week} \\ s_f + 1 & \text{otherwise} \end{cases}$$

Output Definition

At the end of each week, the system produces:

1. A set of assignments `fellow_id` $\rightarrow$ `slot_id`, such that every assigned fellow's slot belongs to their available slot list and is owned by a mentor whose specialties include the fellow's requested topic.
2. For every fellow who submitted a request but was not assigned, a classification of either **infeasibly unserved** or **algorithm failure**, defined as follows. Let C_f denote the set of slots that satisfy both the topic match and availability constraints for fellow f :
 - **Infeasibly unserved:** $C_f = \emptyset$. No compatible slot exists for fellow f this week regardless of what other fellows are assigned. The algorithm cannot be faulted for this outcome.
 - **Algorithm failure:** $C_f \neq \emptyset$, yet fellow f remains unassigned while at least one slot in C_f is left unoccupied in the final assignment. A compatible slot was available and unused, so the failure is attributable to the algorithm.

Assumptions

- A fellow submits at most one request per week.
- A slot holds exactly one fellow (one-on-one).
- Mentor specialties and slot offerings are fixed and fully known at the start of each week.
- Requests from different fellows may compete for the same slot. Being outcompeted for a slot is not an algorithm failure.

Constraints

1. **Topic match:** a fellow may only be assigned to a slot whose mentor lists the requested topic among their specialties.
2. **Availability:** the assigned slot must appear in the fellow’s available slot list.
3. **Slot capacity:** each slot is assigned to at most one fellow per week.
4. **One slot per fellow per week:** each fellow receives at most one assignment per week.

Objective Function

The algorithm minimizes the total starvation penalty for the current week:

$$\text{Penalty}_{\text{week}} = \sum_{f \in U} w(\text{urgency}_f) \cdot (s_f + 1)^2$$

where U is the set of fellows who submitted a request but were not assigned this week, and $(s_f + 1)$ is the value s_f will take after the update. The urgency weights are:

$$w(\text{blocker}) = 3, \quad w(\text{normal}) = 2, \quad w(\text{exploratory}) = 1$$

The algorithm minimizes $\text{Penalty}_{\text{week}}$ independently each week. The accumulated total across all weeks:

$$\text{Penalty}_{\text{total}} = \sum_{t=1}^T \text{Penalty}_t$$

is reported as a performance metric at the end of the program. Each week’s contribution is a fresh charge for that week’s assignment decision. It is not a re-counting of previous weeks, but a new cost reflecting how much it hurts to skip a fellow given how long they have already been waiting.

Tie-Breaking Rules

When multiple feasible assignments yield the same weekly penalty, ties are broken in the following order:

1. Higher urgency tier first (**blocker** > **normal** > **exploratory**).
2. Higher current s_f value.
3. Earlier request timestamp.

Sample Cases

Case 1: Easy

All R fellows submit requests. Every requested topic is covered by at least one available mentor slot, and the number of slots S is large enough that every fellow can be accommodated. All fellows begin with $s_f = 0$.

Expected result: Every fellow is assigned. $\text{Penalty}_{\text{week}} = 0$.

Case 2: Topic Bottleneck

R fellows submit requests. A majority request the same topic, but only a small number of mentors cover it, offering far fewer slots than there are requests for that topic. The remaining fellows request topics with sufficient coverage.

This case tests whether the algorithm:

- never assigns a fellow to an off-topic slot,
- selects which fellows to serve for the bottleneck topic using the urgency-and-starvation objective,
- correctly increments s_f for unserved fellows and classifies each as infeasibly unserved or algorithm failure.

Case 3: Starvation Rescue

One slot remains for a given topic, contested by two fellows:

	Fellow A	Fellow B
Urgency	exploratory ($w = 1$)	blocker ($w = 3$)
Current s_f	3	0
Penalty if unassigned	$1 \cdot (3 + 1)^2 = 16$	$3 \cdot (0 + 1)^2 = 3$

Since $16 > 3$, assigning Fellow A minimizes the weekly penalty. The algorithm should therefore assign Fellow A despite Fellow B having higher urgency. This is a deliberate stress test of the quadratic starvation term: it verifies that a sufficiently starved fellow can outweigh a higher-urgency competitor.

Successful Solution Criteria

An algorithm's output is considered correct if all of the following hold:

1. **No hard constraint is violated:** topic matching, availability, slot capacity, and the one-slot-per-fellow rule are all satisfied.
2. **No algorithm failures occur:** every fellow with a non-empty compatible slot set C_f and at least one unoccupied slot in C_f is assigned.
3. **Near-optimal penalty:** on small instances where exhaustive enumeration of all feasible assignments is tractable, the total weekly penalty produced by the algorithm is within 10% of the minimum achievable penalty.